diffmrc Manual

Reference, methods, and verification records

Version 0.1

The reference document for the diffmrc package: installation and configuration, the API surface, protocol details, the design of the differentiable chain and its second derivatives, the verification records, benchmark results, and the development architecture. The companion *Tutorial* covers first use.

Contents

I User reference **3**

1 Overview and package layout **4**
1.1 Design principles 4
1.2 Package layout 4
1.3 How the documents are organized 5

2 Installation and configuration **6**
2.1 Supported environment 6
2.2 Runtime configuration 6
2.3 Verifying the installation 6
2.4 Optional: running the figures 7
2.5 Known environment friction 7

3 API reference **8**

4 Protocols and options **9**

II Methods and records **10**

5 The differentiable chain **11**

6 Second derivatives **12**

7 Verification records **13**

8 Benchmarks **14**

9 Known limitations **15**

III Development **16**

10 Architecture and contracts **17**

11 Reproducing the paper **18**

Part I

User reference

Chapter 1

Overview and package layout

1.1 Design principles

Three commitments shape the package, and most of its structure follows from them.

The first is that derivatives are never written by hand. Every nuclear gradient and second derivative is obtained by differentiating the energy code path with JAX. This is what makes the CASSCF, state-averaged CASSCF, and NEVPT2 layers share one implementation of the physics: a correction to the energy expression propagates to all of its derivatives without a separate derivation.

The second is that the package is layered, and the layer boundaries are tested. The kernel cannot import the layers above it; the analysis layer cannot import the kernel at all. The contract is enforced in the test suite (see Chapter **??**), so a violation is a failing test rather than a matter of review discipline.

The third is that convergence is reported, not assumed. Optimizers return the gradient norm at the point they stopped at, and the package distinguishes "the optimizer stopped improving" from "the point is stationary." Callers are expected to carry that distinction into whatever they claim about a result.

1.2 Package layout

```
src/diffmrc/
  core/       numerical kernel: rotations, CASCI, CASSCF, NEVPT2,
              stationary points, the frozen source of the chain
  analysis/   post-processing that reads converged objects: entropies,
              mutual information, spin and state identification,
              orbital identity and frontier windows; NumPy only
  policy/     selection rules over analysis output: composition,
              entropy-based and ranked orbital selection, verification
  driver/     composition layer: assembles the kernel into the
              differentiable SCF -> CASSCF -> NEVPT2 pipelines,
              orbital-point convergence, frozen-geometry factories
  report/     result objects and rendering
  cli/        the command-line entry point
```

The dependency direction runs downward only: `core` at the bottom, `cli` at the top. A user script may import any layer; the layers themselves may not import upward. This is what allows the analysis layer to be tested and used with nothing but NumPy installed.

1.3 How the documents are organized

This manual is the reference. Part I covers installation, the API surface, and the protocol details that the code reports but does not enforce. Part II carries the methods: the design of the differentiable chain, the second-derivative construction, the verification records against independent implementations, the benchmark results, and the consolidated limitation list. Part III is for development: layer contracts, the test architecture, and the reproduction steps for the paper.

The companion *Tutorial* is the on-ramp: installation, a first CASSCF gradient, state-averaged workflows, active-space selection, and a full benchmark walkthrough, each with runnable code.

Chapter 2

Installation and configuration

2.1 Supported environment

The package is developed and verified on Linux and Windows with CPython 3.11, on CPU. The Tutorial (Chapter 2) gives the minimal recipe; this chapter records the constraints behind it.

Dependency	Constraint	Why it is pinned
Python	≥ 3.10	`int.bit_count` in the determinant tools
numpy	≥ 1.24	analysis layer, reference arithmetic
scipy	≥ 1.10	L-BFGS-B and trust-region stages
pyscfad	$\geq 0.3.3$	provides the traceable SCF and integral path
pyscf	≥ 2.5	reference energies and comparisons
jax	≥ 0.4	array layer; the exact pin is pyscfad's
matplotlib	any recent	figure sources only

`pyscfad` pins a specific JAX version, which is why the package should live in its own environment. Mixing it into an environment with a different JAX pin produces failures that look like numerical bugs and are not.

2.2 Runtime configuration

Three settings are load-bearing.

`jax_enable_x64 = True`. The package is validated in double precision. With the default single precision the errors are not loud: gradients lose several digits and some convergence gates become unreachable, while energies look plausible.

`JAX_PLATFORMS = cpu`. The CPU platform is the verified configuration. On a machine with a CUDA device, leaving the platform unset selects the GPU, where the solver path used by the orbital canonicalization raises an error unrelated to the chemistry. Set the variable before importing JAX; setting it afterwards has no effect.

Thread environment variables. The driver assumes it owns the thread budget it was given. Setting the five BLAS thread variables keeps multiple concurrent calculations from oversubscribing a node; their absence does not corrupt results, it makes timing meaningless and slows everything.

2.3 Verifying the installation

```
pytest tests/ -v
lint-imports        # or: pytest tests/test_layer_contracts.py
```

The suite is the primary artifact to look at: it contains the first-order and second-order derivative checks against finite differences, the cross-validation against PySCF, and the layer contracts. A passing suite on your machine is the intended proof that the environment is sound.

2.4 Optional: running the figures

The figure sources under `figures/` import matplotlib and read JSON results produced by the scripts; they require no additional configuration. They are not part of the package tests.

2.5 Known environment friction

- The first evaluation of a freshly built node of the chain triggers XLA compilation that can take minutes. This is expected; a stopwatch on a single evaluation will mislead.
- pyscfad's `Mole` objects cannot cross a `jax.jit` boundary. The driver layer offers frozen-geometry factories for jitted contexts; the factory docstrings state which is which.
- On shared nodes, leave headroom for the owner of the other processes. The memory profile of this package is dominated by compilation- and evaluation-time buffers of the differentiated chain, not by the integrals, and it grows steeply with the number of rotation parameters.

Chapter 3

API reference

Chapter 4

Protocols and options

Part II

Methods and records

Chapter 5

The differentiable chain

Chapter 6

Second derivatives

Chapter 7

Verification records

Chapter 8

Benchmarks

Chapter 9

Known limitations

Part III

Development

Chapter 10

Architecture and contracts

Chapter 11

Reproducing the paper